

Programming for Robotics: Project Report

Project Title: Voom-Bot: Autonomous Indoor Cleaning Robot

GitHub link: <https://github.com/SpookySaikiK/3003ICT-Group-7-Project>

Group Members:

- Lucas El-Harrif – s5342319
- Michelle Murutsi – s5299221
- Tashinga Blessed Nyamadzawo – s5415355

Specialisation Option Chosen: Autonomous Robot (Webots)

1. Project Overview

Voom-Bot is an autonomous indoor cleaning robot developed in Webots using the Pioneer 3-DX platform. The purpose of the robot is to autonomously explore, map, and clean indoor environments such as classrooms, offices, libraries, and living spaces while safely avoiding hazards and obstacles. The robot solves the problem of autonomous floor coverage in unknown indoor environments without requiring manual control. It performs exploration, obstacle avoidance, cleaning coverage, cliff detection, docking, and battery management using a structured finite state machine (FSM). The system integrates multiple sensors and actuators to demonstrate intelligent robotic behaviour.

Component	Implementation
Sensors	16 proximity sensors, cliff sensors, GPS, compass, keyboard
Actuators	Wheel motors, LEDs, speaker
Behaviours	Wall-following, obstacle avoidance, docking, cliff protection, recovery, grid cleaning

The robot operates inside a simulated indoor Webots environment with walls, obstacles, and drop off hazards.

2. Technical Requirements Compliance

Core Requirements

Requirement	Implementation
At least 2 inputs	Proximity sensors, cliff sensors, GPS, compass, and keyboard input
At least 2 outputs	Wheel motors, LEDs, and speaker
FSM or Behaviour Tree	Finite State Machine implemented with 7 states
Minimum 4 Behavioural states	PAUSED, EXPLORE, CLEANING, AVOID, CLIFF, RECOVERY, CHARGING
Multi-condition decision logic	State transitions depend on battery level, obstacle detection, cliff detection, exploration completion and user input
Safety or fail-safe mechanism	Cliff avoidance, obstacle avoidance, low battery charging, stuck detection, recovery

Autonomous Robotics Requirements (Option B)

Requirement	Implementation
Navigation	GPS and compass based navigation with grid target movement
Obstacle Avoidance	Reactive turning using proximity sensor data
Perception-driven decisions	Sensor readings directly influence state transitions and robot behaviour

3. Behaviour Model

The robot behaviour is controlled using a Finite State Machine (FSM). FSMs were chosen because they provide clear behavioural separation, predictable transitions, and simplified debugging.

Behavioural States

State	Purpose
PAUSED	Robot remains Idle
EXPLORE	Wall following exploration
CLEANING	Grid based coverage
AVOID	Reactive obstacle avoidance
CLIFF	Emergency cliff avoidance
RECOVERY	Handles stuck situations
CHARGING	Returns to dock and recharges

The FSM structure improves modularity and ensures safety critical behaviours override normal operation.

4. Perception / Sensor Processing

The robot combines multiple sensor inputs to make intelligent navigation decisions.

System	Function
Obstacle sensing	Weighted proximity sensors determine turning direction
Cliff sensing	Cliff sensors trigger reverse and rotate behaviour
Localisation	GPS and compass provide position and heading
Mapping	Grid cells track visited/unvisited areas
Decision logic	FSM combines sensor data, battery level, and safety conditions

5. Safety and Robustness

Several safety mechanisms were implemented to improve reliability.

Safety Mechanism	Behaviour
Obstacle avoidance	Robot turns away from nearby objects
Cliff protection	Reverse and rotate away from edges
Battery protection	Return-to-dock charging
Stuck detection	Recovery state with speaker alerts
Fail-safe override	Safety states override cleaning/navigation

6. Code Structure

The project is divided into modular Python files to improve readability and maintainability.

File	Purpose
main_controller.py	Main control loop and FSM logic
config.py	System constants and configuration values
sensors.py	Sensor initialisation and processing
navigation.py	Navigation and movement behaviours
mapping.py	Grid mapping and coverage logic
utils.py	Shared utility functions

Main Control Loop

The controller repeatedly performs:

1. Sensor updates
2. State transition checks
3. Behaviour execution
4. Motor control output
5. Battery updates

This structure follows standard robotic control architecture.

Key Libraries Used

- math
- controller

7. Key Code Snippets

FSM State Transition Logic

```
if battery <= BATTERY_LOW and state not in (CHARGING, RECOVERY, CLIFF, AVOID):
    charging_complete = False
    state = CHARGING
```

This logic ensures the robot safely returns to the dock when battery becomes low.

Obstacle Detection Processing

```
for i, sensor in enumerate(sensors):
    value = sensor.getValue()
    if value <= 0:
        continue

    distance = 5.0 * (1.0 - value / 1024.0)
    if distance >= 0.5:
        continue

    influence = 1.0 - (distance / 0.5)

    wl, wr = WEIGHTS[i]
    left_force += wl * influence
    right_force += wr * influence
```

This weighted perception system converts sensor readings into directional turning behaviour.

Cliff Detection Safety Override

```
if cliff_detected and state != CLIFF and state != RECOVERY:
    state = CLIFF
```

Cliff detection immediately overrides all other behaviours to protect the robot.

Target Navigation

```
target_angle = math.atan2(dz, dx)
angle_error = wrap_angle(target_angle - theta)

if abs(angle_error) > 0.2:
    turn = clamp(KP_ANGLE * angle_error, -MAX_SPEED, MAX_SPEED)
    return -turn, turn, False
```

This code aligns the robot toward cleaning targets using proportional heading control.

8. Results / Demonstration Summary

The robot successfully demonstrated:

- Autonomous exploration of indoor environments
- Reactive obstacle avoidance
- Cliff detection and prevention
- Grid based cleaning coverage
- Automatic return to dock charging
- Battery aware behaviour transitions
- Stuck detection and recovery alerts

The FSM produced stable behaviour transitions and reliable autonomous operation.

Limitations

- The robot uses simple wall following instead of full SLAM navigation.
- Mapping accuracy depends on GPS precision and cell size configuration.

Challenges Encountered

- Balancing obstacle avoidance responsiveness
- Preventing oscillation during wall following
- Managing smooth transitions between exploration and cleaning state

10. Individual Contributions

Member	Contribution
Lucas El-Harrif	FSM implementation, battery system, docking logic, documentation, obstacle avoidance, mapping system
Michelle Murutsi	FSM implementation, testing, debugging, sensor processing, documentation
Tashinga Blessed Nyamadzawo	FSM implementation, testing, debugging, video demonstration, documentation, mapping system